

THE UNIVERSITY OF ZAMBIA
SCHOOL OF NATURAL AND APPLIED SCIENCES
DEPARTMENT OF COMPUTING AND INFORMATICS

FINAL YEAR PROJECT REPORT

ACADEMIC YEAR 2026

**SVDS: A Camera-Based Stolen Vehicle Detection
System for the Zambia Police Service — The AI/Vision
Component (Plate Detection Model and Camera Node)**

A Final Year Project Report Submitted to the
Department of Computing and Informatics
in partial fulfilment of the requirements for the award of

[DEGREE PROGRAMME — confirm]

Student Name: Moomba Nicholas Katapazi
Computer Number: [COMPUTER NUMBER]
Supervisor: [SUPERVISOR NAME]
Date of Submission: [DATE OF SUBMISSION]

DECLARATION

I declare that this final year project report is my own work and has been prepared in accordance with the academic requirements of the University of Zambia. All sources of information, ideas, data, software, images, and other material that are not my own have been appropriately acknowledged and referenced.

Where this project was undertaken as part of a group, I have clearly identified my individual contributions and have not presented the work of other group members as my own.

Student Name: Moomba Nicholas Katapazi

Computer Number: _____

Signature: _____

Date: _____

APPROVAL

This final year project report has been submitted for examination with the approval of the project supervisor.

Supervisor Name: _____

Signature: _____

Date: _____

Departmental/Examiner **Approval** **(if** **applicable):**

ABSTRACT

Stolen-vehicle recovery in Zambia depends largely on police officers chancing to recognise a plate number from a paper report or radio bulletin. This project, carried out by a group of four students, built SVDS, a Stolen Vehicle Detection System on top of the VRRS vehicle registry. The public files stolen-vehicle reports through a web application, and a camera node reads plates from a phone's video stream and checks them against active reports, so connected police officers are alerted in real time.

This individual report covers the AI/vision component: the plate-detection model and the camera node. A single-class YOLOv8n detector was fine-tuned on a Roboflow-annotated dataset of 1,890 labelled images (1,476 training, 407 validation, 7 test) of Zambian number plates. Detected plates are cropped, converted to greyscale, upscaled and thresholded, and read with EasyOCR restricted to letters and digits. The text is validated as 5–8 alphanumeric characters and then de-duplicated by fuzzy string similarity, because OCR noise defeats exact-match cooldowns. Each accepted reading is sent to the backend over a single HTTP endpoint, so the node runs as a decoupled process.

After 100 training epochs the model reached a precision of 0.988, a recall of 0.751, an mAP50 of 0.801 and an mAP50-95 of 0.686 on the validation split. The eight unit tests for the node's plate-cleaning and de-duplication logic all pass. A live run of the node read plates from a phone stream and reached the backend, but reading accuracy (OCR), alert latency and a recorded stolen-vehicle alert were not measured, and the report states these limitations plainly. The main contributions are a locally trained plate detector, a noise-tolerant read-and-report pipeline, and a decoupled camera-node design that could be moved to fixed roadside cameras without changing the backend.

Keywords: licence plate recognition; YOLOv8; object detection; optical character recognition; stolen vehicle detection

ACKNOWLEDGEMENTS

[Write acknowledgements here: supervisor, group members (backend and security, real-time and integration, frontend), family, and anyone who helped collect plate images. The author must write this; it is personal.]

CONTENTS

Preliminary pages: Declaration · Approval · Abstract · Acknowledgements · List of Abbreviations and Acronyms · Note on Group Projects

1 Introduction — 1.1 Background and Context · 1.2 Problem Statement · 1.3 Project Aim · 1.4 Project Objectives · 1.5 Research Questions · 1.6 Scope of the Project (In Scope, Out of Scope) · 1.7 Significance · 1.8 Limitations · 1.9 Organisation of the Report

2 Literature Review — 2.1 Introduction · 2.2 Key Concepts and Theories · 2.3 Existing Systems / Related Work · 2.4 Comparative Analysis · 2.5 Research / Knowledge Gap · 2.6 Conceptual Framework · 2.7 Chapter Summary

3 Methodology — 3.1 Introduction · 3.2 Research / Development Approach · 3.3 Requirements / Data Collection · 3.4 System Architecture / Research Workflow · 3.5 Tools and Technologies · 3.6 Ethical and Legal Considerations · 3.7 Evaluation Method · 3.8 Chapter Summary

4 System Analysis and Design — 4.1 Introduction · 4.2 Stakeholder and User Analysis · 4.3 Functional Requirements · 4.4 Non-Functional Requirements · 4.5 Use Cases / User Stories · 4.6 System / Solution Architecture · 4.7 Database / Data Design · 4.8 Interface / Interaction Design · 4.9 Algorithms / Models / Technical Design · 4.10 Chapter Summary

5 Implementation / Development — 5.1 Introduction · 5.2 Development Environment · 5.3 System Components / Modules · 5.4 Key Implementation Details · 5.5 Integration · 5.6 Challenges and Solutions · 5.7 Individual Contribution · 5.8 Chapter Summary

6 Testing, Results and Evaluation — 6.1 Introduction · 6.2 Testing Strategy · 6.3 Test Cases and Results · 6.4 Experimental / Performance Results · 6.5 User Evaluation · 6.6 Analysis of Results · 6.7 Objective-by-Objective Evaluation · 6.8 Individual Results · 6.9 Chapter Summary

7 Discussion — 7.1 Introduction · 7.2 Interpretation of Findings · 7.3 Comparison with Existing Work · 7.4 Contributions of the Project · 7.5 Limitations and Implications · 7.6 Chapter Summary

8 Conclusion and Recommendations — 8.1 Introduction · 8.2 Summary of the Project · 8.3 Conclusions · 8.4 Achievement of Objectives · 8.5 Recommendations · 8.6 Future Work

References

Appendices — A Individual Contribution Statement · B User Manual / Installation Guide · C Additional System Designs / Diagrams · D Additional Test Cases and Results · E Data Collection

Instruments · F Additional Code / Configuration · G Other Supporting Material

[Page numbers are not shown here. In Word, apply heading styles and use References → Table of Contents to generate a numbered contents page, and a List of Figures and List of Tables the same way.]

LIST OF ABBREVIATIONS AND ACRONYMS

Abbreviation	Meaning
ALPR	Automatic Licence Plate Recognition
API	Application Programming Interface
CUDA	Compute Unified Device Architecture (NVIDIA GPU computing platform)
DBMS	Database Management System
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IoU	Intersection over Union
JWT	JSON Web Token
mAP	mean Average Precision
MJPEG	Motion JPEG (video stream format)
ML	Machine Learning
OCR	Optical Character Recognition
SVDS	Stolen Vehicle Detection System
UI	User Interface
VRRS	Vehicle Registry and Reporting System
YOLO	You Only Look Once (object detector family)

NOTE ON GROUP PROJECTS

Students who worked on a project as a group are strongly encouraged to prepare their own individual final reports rather than submitting identical reports. The project, system, dataset, or overall research problem may be shared, but each student's report should clearly demonstrate that student's own understanding, responsibilities, work, results, and contribution to the project.

In particular, group-project students should:

- state the overall group project and explain how the individual work fits within it;
- clearly identify their own responsibilities and deliverables;
- focus detailed analysis, design, implementation, experiments, testing, and/or evaluation on their own contribution;
- acknowledge the work completed by other group members where it is discussed;
- include an *Individual Contribution Statement* in the appendices; and
- not claim another group member's work, results, code, data collection, or design decisions as their own.

Chapter 1: Introduction

1.1 Background and Context

Vehicle theft is a persistent problem for law enforcement in Zambia. Recovering a stolen vehicle depends heavily on chance: an officer at a checkpoint or on patrol happens to recognise a plate number from a paper report or a radio bulletin. Vehicle records, incident reports and eyewitness descriptions are typically kept on paper or in disconnected station-level spreadsheets. A report filed at one police station is therefore invisible to an officer working in a different part of the same city, let alone a different province.

At the same time, automatic licence plate recognition (ALPR) has become cheap enough to build from commodity hardware. A single Android phone can stream video over Wi-Fi, and open-source object detectors such as Ultralytics YOLOv8 can be trained on a modest, self-collected image set and run in real time on a consumer GPU. This closes the gap between "a report exists somewhere" and "a report is automatically checked against every vehicle a camera sees".

This report describes **SVDS**, the Stolen Vehicle Detection System, built as a group project on top of the **VRRS** (Vehicle Registry and Reporting System) web application. The group built one system with four areas of responsibility (see the Note on Group Projects and Appendix A). This report is the individual report of the student responsible for the **AI/vision component**: the plate-detection model and the camera node that turns a live video stream into plate readings.

1.2 Problem Statement

The Zambia Police Service has no system that automatically checks the licence plate of a vehicle, as seen by a camera in the field, against an active stolen-vehicle registry in real time. Existing practice depends on manual recognition by officers and manual cross-referencing of paper reports. The consequences are:

1. slow or missed detection of stolen vehicles that pass through monitored locations;
2. no centralised, real-time route by which a report filed by the public reaches every officer who might encounter the vehicle; and
3. no audit trail linking a detection event back to the report that triggered it.

The specific problem addressed by this individual report is the first link in that chain: **reliably turning a phone-camera video stream of Zambian number plates into clean plate strings that the rest of the system can match against the registry**, using only inexpensive hardware and no paid recognition service.

1.3 Project Aim

Group aim. To design, implement and evaluate SVDS: a vehicle registry and reporting web application integrated with a real-time, camera-based licence plate detection pipeline, for use by the Zambia Police Service and the public it serves.

Individual aim (AI/vision). To build and evaluate the detection model and camera-node pipeline that detects Zambian licence plates in a live video stream, reads them, and reports them to the SVDS backend.

1.4 Project Objectives

Group objectives

1. Implement role-based accounts (Reportee, Police, Admin) with access scoped to each role.
2. Let the public file and track stolen-vehicle reports (under_review → missing → found).
3. Give police officers a live view of detection alerts, the registry and analytics.
4. Give administrators control of accounts, system health and the audit trail.
5. Detect and read plates from a live camera feed and match them against active reports.
6. Deliver an alert to connected police clients over WebSocket the moment a stolen vehicle is detected.

Individual objectives (AI/vision)

No.	Objective	Measure of success
O1	Collect and annotate a dataset of Zambian licence plates	A YOLO-format dataset with train, validation and test splits
O2	Train a single-class YOLOv8 plate detector	Precision above 0.95 and mAP50 above 0.75 on the validation split
O3	Read the plate text from each detected region using OCR and validate its format	Output limited to 5–8 alphanumeric characters; malformed reads discarded
O4	Suppress repeated reports of the same plate despite OCR noise	Near-identical readings inside a cooldown window produce one report
O5	Build a decoupled camera node that streams frames, runs O2–O4 and reports to the backend over HTTP	Node runs as its own process and recovers from stream loss
O6	Evaluate the model and node quantitatively and through automated tests	Precision, recall and mAP reported; unit tests pass

1.5 Research Questions

1. Can a custom-trained, lightweight detector (YOLOv8n) combined with an open-source OCR engine detect and read Zambian number plates well enough for real-time matching, using only a phone camera and a consumer GPU?
2. Can that pipeline be integrated into a role-based registry as a fully decoupled component that talks to the backend only over plain HTTP?

(A third question, about the latency benefit of WebSocket alerts, belongs to the real-time and integration member's report and is not answered here.)

1.6 Scope of the Project

1.6.1 In Scope

- **Group:** the VRRS/SVDS web application (backend, database, frontend), the camera node, and the integration between them.
- **This report:** dataset collection and annotation, detector training and evaluation, the OCR and post-processing pipeline, the fuzzy duplicate-suppression logic, the camera-node process (vrrs-node), and the unit tests for the node's pure logic.

1.6.2 Out of Scope

- A native mobile app for the public (the public interface is a responsive web app).
- Payment or e-government integration.
- Fleet management of many camera nodes (one configured node is supported).
- GPS-based camera geolocation: the camera's location is a configured string.
- Recognition of plate formats other than the Zambian plates in the training set, and recognition of vehicle make, model or colour.

1.7 Significance of the Project

The project benefits the Zambia Police Service, which gains a way to check every plate a camera sees against the stolen-vehicle registry without an officer having to recognise it, and the public, whose reports become actionable immediately. Technically, it shows that a useful ALPR pipeline can be built from a phone, a consumer GPU and open-source software, with a model trained on locally collected data instead of a paid API. Because the camera node talks to the backend only over HTTP, the same pipeline could later be pointed at fixed roadside cameras without changing the backend.

1.8 Limitations

- **Dataset size and provenance.** The dataset is small and comes from a single annotation project. The Roboflow export README states "335 images", while the exported splits on disk contain 1,890 images (§3.3). The number of original source photographs is therefore not established by the records available and must be confirmed before submission.
- **No recorded *STOLEN* alert.** A live run of the node against the phone stream exists (eight readings, all answered CLEAR), but no run in which a real vehicle matched an active report and raised a dashboard alert was recorded (§6.4).
- **Hardware.** Real-time performance was only measured on one machine (NVIDIA Quadro P2000). CPU-only operation falls behind a live stream.
- **Small test split.** The held-out test split has only seven images, so test-split figures are weak evidence.
- **No plate-text ground truth.** The dataset labels plate *locations*, not plate *text*, so end-to-end OCR accuracy has not been measured (§6.6).
- **Time.** The remaining known issues of the wider system are listed in Chapter 8.

1.9 Organisation of the Report

- **Chapter 2** reviews ALPR concepts, existing systems and the gap this project fills.
- **Chapter 3** describes the development approach, tools, ethics and evaluation method.
- **Chapter 4** gives the analysis and design: requirements, architecture and the detection algorithms.
- **Chapter 5** describes the implementation, with emphasis on the AI/vision component and the student's individual contribution.
- **Chapter 6** presents testing, results and an objective-by-objective evaluation.
- **Chapter 7** discusses the findings and their limits.
- **Chapter 8** concludes and gives recommendations and future work.
- **Appendices** hold the Individual Contribution Statement, the installation guide, extra designs and results, and selected code.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the concepts behind automatic licence plate recognition (ALPR), the existing systems relevant to SVDS, and the gap the project fills. It is organised as: key concepts (§2.2), existing systems (§2.3), a comparison (§2.4), the gap (§2.5), and the framework that guides the design (§2.6).

2.2 Key Concepts and Theories

2.2.1 Two-stage ALPR

Most ALPR pipelines split the problem into two stages: (1) **localisation**, which finds the plate in the image, and (2) **recognition**, which reads the characters inside the located region. Keeping the stages separate makes each one simpler, independently testable and replaceable.

2.2.2 Object detection and YOLO

Object detection predicts a bounding box and a confidence score for each object of interest in an image. The YOLO ("You Only Look Once") family are single-stage detectors: one network pass produces all boxes, which makes them fast enough for video. YOLOv8, from Ultralytics, is an anchor-free member of the family with several sizes; the smallest, **YOLOv8n**, is used here because it runs in real time on a modest GPU. Models are normally fine-tuned from weights pretrained on a large dataset (COCO) rather than trained from scratch, which reduces the amount of new data needed.

2.2.3 Optical character recognition

OCR converts an image of text into a string. **EasyOCR** is an open-source, PyTorch-based engine that supports a character *allowlist*, which restricts the output alphabet and reduces spurious characters. OCR accuracy on a small plate crop depends strongly on pre-processing such as greyscale conversion, upscaling and binarisation (thresholding). **Otsu's method** chooses the binarisation threshold automatically from the image histogram.

2.2.4 Detection metrics

- **Precision** = $TP / (TP + FP)$: of the boxes the model draws, the fraction that are correct.
- **Recall** = $TP / (TP + FN)$: of the real plates, the fraction the model finds.
- **IoU** (intersection over union) measures how well a predicted box overlaps a true box.

- **mAP50** is the mean average precision when a prediction counts as correct at $\text{IoU} \geq 0.5$; **mAP50-95** averages over IoU thresholds from 0.5 to 0.95 and is stricter.

2.2.5 Fuzzy string matching

OCR on a moving or stationary vehicle rarely returns exactly the same string in consecutive frames. Comparing readings by *similarity* instead of equality tolerates this. Python's `difflib.SequenceMatcher` returns a similarity ratio between 0 and 1 for two strings; it is used here to decide whether a new reading is the same plate as a recent one.

2.3 Existing Systems / Related Work

- **OpenALPR** is an open-source ALPR library with pre-trained models for several countries' plate formats. It is a strong general reference, but it does not ship a model trained on Zambian plates.
- **Plate Recognizer** is a commercial ALPR service. It uses the same split as this project, a lightweight capture client plus a separate recognition service. It is a paid API, which is a barrier for a budget-limited public-sector deployment.
- **Ultralytics YOLOv8** is chosen here over older YOLO versions and over two-stage detectors such as Faster R-CNN for its balance of speed and accuracy on one consumer GPU, and for its support for training on a small custom single-class dataset.
- **EasyOCR** is used for recognition (§2.2.3).
- **Vehicle registry and reporting systems.** Registry and reporting systems store reports, ownership records and status but do not themselves watch for the reported vehicle. Where ALPR is used in policing, it is typically fixed roadside infrastructure rather than something integrated with a public reporting system.

Note on sources. The descriptions above are based on the projects' public documentation, listed in the References. No comparative accuracy figures from other systems are quoted, because none were measured on Zambian plates in this project.

2.4 Comparative Analysis

Criterion	OpenALPR	Plate Recognizer	This project (SVDS)
Cost model	Open source	Paid service	Open source, self-hosted
Trained on Zambian plates	No (generic regional models)	Not specifically	Yes (custom dataset)
Runs on own hardware	Yes	Cloud API	Yes (phone + local GPU)

Criterion	OpenALPR	Plate Recognizer	This project (SVDS)
Linked to a public reporting workflow	No	No	Yes (matches against active reports)
Alerts to officers in real time	Not included	Not included	Yes (WebSocket, built by the integration member)

The table compares design properties, not accuracy. No accuracy comparison was carried out.

2.5 Research / Knowledge Gap

No system reviewed combines (a) public self-service reporting, (b) role-based police and admin oversight, (c) a detector trained for the specific plate format, and (d) real-time delivery of detection alerts. SVDS fills that gap. For the AI/vision component the specific gap is a **detector trained on locally collected Zambian plate images and a robust, noise-tolerant read-and-report pipeline around it**.

2.6 Conceptual Framework

The design follows a staged pipeline in which each stage discards input it cannot trust, so only clean, validated readings reach the backend:

```
frame → sample every Nth frame → detect plate (YOLOv8n) → crop
      → greyscale / 4x upscale / Otsu threshold → OCR (allowlist)
      → clean and validate text → fuzzy duplicate check → report to backend
```

Localisation is separated from recognition (§2.2.1), and the camera node is separated from the backend so the two can be deployed and changed independently.

2.7 Chapter Summary

ALPR and vehicle-registry systems are both mature on their own, but no reviewed system joins them for a Zambian plate format with a public, police and admin role split. The AI/vision component therefore adopts a two-stage pipeline: a custom-trained YOLOv8n localiser, EasyOCR recognition with pre-processing and validation, and fuzzy duplicate suppression, run in a decoupled camera-node process.

Chapter 3: Methodology

3.1 Introduction

This chapter explains how the project was carried out: the development approach, how requirements and data were obtained, the architecture, the tools, the ethical considerations, and how success was measured.

3.2 Research / Development Approach

SVDS was built **iteratively**, not to a single up-front specification. The web application (backend and frontend) and the camera-node detection pipeline ran as two work-streams that meet at one integration point, the `POST /alerts/check-plate` endpoint. This let the model's cycle (collect data, annotate, train, evaluate) proceed without waiting on the CRUD and authentication work, and each side could be retested independently.

For the AI/vision component the approach is **experimental**: several training runs were made and compared (the `runs/detect/` directory holds runs named `svds-plate-detector`, `-2`, `-3`, `-final`, `-final-2` and others). The run `svds-plate-detector-final-2` produced the weights used by the camera node. The pipeline around the model was then refined against real footage from a phone camera, not only against the still-image validation set.

Requirements → Design → Implementation → Integration → Testing → (issues found? back to Requirements)

3.3 Requirements / Data Collection

Requirements were derived from (a) the gap in Chapter 1, (b) the natural three-way split of who touches a stolen-vehicle report (the person who filed it, the officer investigating, the administrator), and (c) the technical constraints of detection: a single cheap camera source, and no coupling of the detector to the backend's release cycle. No formal stakeholder interviews are recorded for this project; the requirements are the group's own analysis of the problem.

Data. Plate images were collected with phone cameras, the same class of device the deployed node uses, so that the training distribution (angle, lighting, blur, resolution) resembles the deployment distribution. They were annotated in Roboflow (project `zambia-number-plate-detection`, version 3) with one bounding-box class, and exported in YOLOv8 format.

Split	Images on disk	Labels on disk
train	1,476	1,476
valid	407	407
test	7	7
Total	1,890	1,890

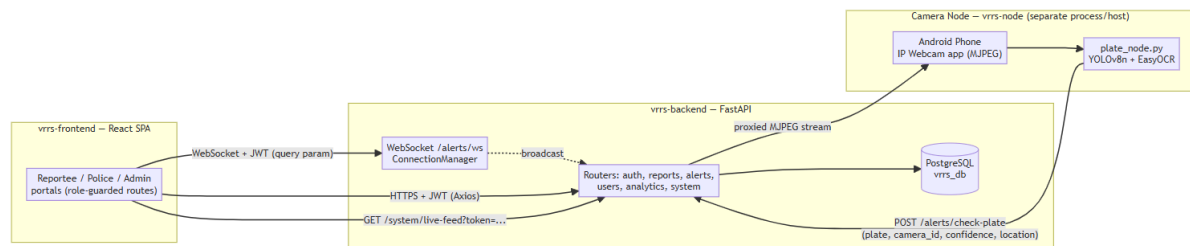
These counts were obtained by counting the files in the exported dataset folder. The Roboflow README that accompanies the export says "The dataset includes 335 images" and describes 3 augmented versions per source image, which would give about 1,005 images and does not reconcile with 1,890. The training-split filenames, which have the form <source-id>_jpg.rf.<hash>.jpg, carry 1,270 distinct source ids for 1,476 images, the validation split 407 for 407, and the test split 7 for 7 (1,684 distinct ids in all). **No source id appears in more than one split**, so augmented copies of a photograph did not leak between training and validation. Because the README count, the 3× augmentation claim and the filename evidence do not agree, **the number of original photographs is left for the student to confirm from the Roboflow project**, and this report does not state it.

Roboflow pre-processing and augmentation, as recorded in the export README:

- auto-orientation (EXIF stripping) and resize to 640×640 (stretch);
- 50% chance of horizontal flip;
- random rotation between -15° and $+15^\circ$;
- random brightness change between -25% and $+25\%$;
- random Gaussian blur of 0–1.8 px (listed in the export README under bounding-box transformations).

3.4 System Architecture / Research Workflow

SVDS consists of three independently runnable processes that communicate only over the network: **vrrs-backend** (FastAPI and PostgreSQL), **vrrs-frontend** (React) and **vrrs-node** (the camera node). The camera node does not import anything from the backend.



The research workflow for the AI/vision component is shown below.



3.5 Tools and Technologies

Area	Technology
Language	Python
Detection	Ultralytics YOLOv8 (yolov8n.pt base weights), PyTorch with CUDA
OCR	EasyOCR
Image and video handling	OpenCV (cv2)
Annotation and augmentation	Roboflow
Node ↔ backend transport	requests (HTTP/JSON)
Configuration	python-dotenv (.env)
Testing	pytest
Hardware	NVIDIA Quadro P2000 GPU (training and inference); Android phone running an IP Webcam app (MJPEG stream)
OS	Windows 11
Rest of system (built by other members)	FastAPI, SQLAlchemy, PostgreSQL, React, Vite

3.6 Ethical and Legal Considerations

- **Personal data in images.** Licence plates identify vehicles, and vehicles can identify owners. Training images should be photographs of vehicles in public places, taken for this project. The dataset should not be published with images that show people's faces. Whether the images were collected with any permission beyond public visibility is not recorded and should be stated by the student.
- **Data minimisation on the node.** The camera node forwards only the plate string, a confidence value, a camera identifier and a configured location string. It does not upload frames, and it holds no database access.
- **Licensing.** The dataset export carries a CC BY 4.0 licence. YOLOv8 (Ultralytics) is released under AGPL-3.0, and EasyOCR under Apache 2.0. Anyone deploying the system beyond an academic prototype needs to check that these licences suit their use.
- **Consequences of a false match.** A wrong plate reading could point officers at an innocent driver. The pipeline therefore favours precision over recall, matches only against reports that police have activated, and leaves the decision to a human officer who sees the alert. Officers can flag false positives in the web application.

- **Endpoint trust.** /alerts/check-plate is unauthenticated because the node has no user identity. It can only return CLEAR or create an alert against an already-active report, so it cannot alter reports. It should nevertheless sit behind a private network or an API key in a real deployment.
- **Security of the rest of the system** (passwords, JWT, role checks) is covered in the backend and security member's report.

3.7 Evaluation Method

1. **Model evaluation:** precision, recall, mAP50 and mAP50-95 from the training run's per-epoch record (results.csv), plus a re-run of the final weights on the validation and test splits.
2. **Unit testing** of the node's pure logic (plate cleaning and fuzzy de-duplication) with pytest, isolated from the GPU stack so the tests can run anywhere.
3. **System observation:** running the node against the live stream and backend and recording what was and was not observed.
4. **Objective-by-objective review** against the individual objectives O1–O6 in §1.4.

No user survey was run; there are no user-evaluation results in this report.

3.8 Chapter Summary

The project followed an iterative approach with an experimental training loop for the model. Data was collected with phone cameras, annotated in Roboflow, and exported as 1,890 labelled images. The tools are open-source Python libraries on a GPU-equipped Windows machine. Evaluation combines model metrics, unit tests and system observation.

Chapter 4: System Analysis and Design

4.1 Introduction

This chapter presents the analysis and design of SVDS. The system-level material (stakeholders, overall architecture) is shared by the group. The detailed requirements, use cases and algorithms are those of the **AI/vision component**, which is the subject of this individual report. Design work on the backend, security, real-time and frontend areas belongs to the other members and is summarised only where the camera node depends on it.

4.2 Stakeholder and User Analysis

Stakeholder	Interest in the system	Interaction with the AI/vision component
Reportee (member of the public)	Files a stolen-vehicle report and follows its status	Indirect: a filed and activated report becomes something the camera can match
Police officer	Investigates reports, receives alerts, marks vehicles found	Receives alerts that the node's readings produce; flags false positives
Administrator	Manages accounts, watches system health and the audit trail	Sees the camera node's online/offline status
Camera-node operator	Mounts the phone, starts the node, edits its <code>.env</code>	Direct user of the node
Zambia Police Service (institution)	Recovered vehicles, an audit trail	Beneficiary

4.3 Functional Requirements

Camera node (AI/vision)

ID	Requirement
FR-N1	Connect to a live MJPEG video stream from a phone-mounted camera.
FR-N2	Process a configurable subset of frames (every Nth frame).
FR-N3	Detect licence plates in a processed frame using the custom-trained detector, at a configurable confidence threshold.
FR-N4	Crop each detected plate and pre-process it for OCR (greyscale, upscale, threshold).
FR-N5	Read the plate text, keep only readings above an OCR-confidence floor, and restrict characters to A–Z and 0–9.

ID	Requirement
FR-N6	Clean and validate the text against the expected plate shape (5–8 alphanumeric characters) and discard anything else.
FR-N7	Suppress repeated reports of the same plate within a cooldown window, tolerating OCR noise between frames.
FR-N8	Send each accepted reading to the backend over HTTP and log the response (CLEAR or STOLEN).
FR-N9	Recover automatically if the video stream drops.
FR-N10	Show a live preview with the detection box and plate text.

System-level requirements the node depends on (built by other members)

- The backend exposes POST /alerts/check-plate, matches the plate against missing reports and returns CLEAR or STOLEN.
- The backend pushes a WebSocket message to connected police clients on a match.
- Role-based access, report lifecycle and analytics are provided by the backend and frontend.

4.4 Non-Functional Requirements

- **Performance:** the node processes every 5th frame by default to keep pace with the stream on the available GPU.
- **Precision over recall:** false detections should be rarer than missed ones, because a single missed frame is recoverable on the next frame but a false alert points officers at an innocent vehicle.
- **Resilience:** reconnect automatically on stream loss; a backend outage must not crash the node (failed POSTs are logged).
- **Decoupling:** no dependency on backend internals; configuration only through environment variables.
- **Testability:** the pure logic (cleaning and de-duplication) must be importable without the GPU stack.
- **Maintainability:** thresholds are configuration, not constants in code.

4.5 Use Cases / User Stories

UC-1: Detect and report a plate

- *Actor:* camera node (system actor); operator starts it.
- *Precondition:* stream reachable, weights present, backend reachable.

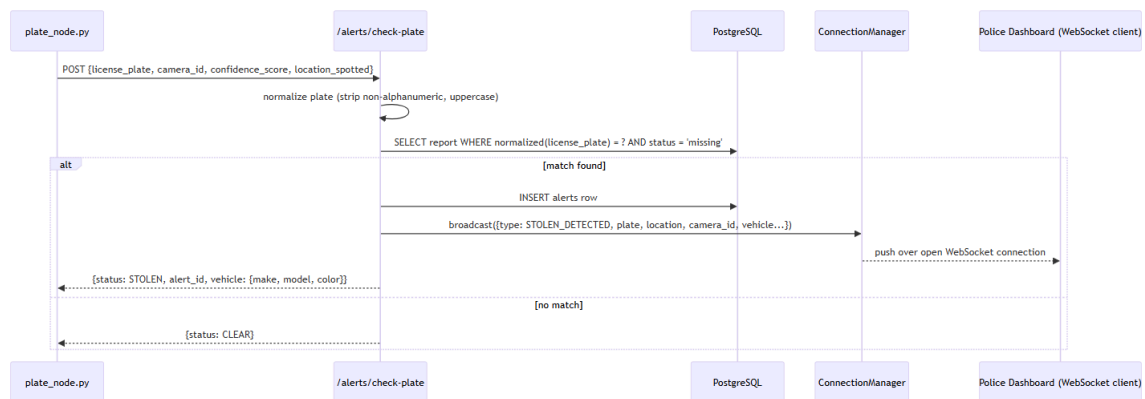
- *Main flow*: the node reads a frame, detects a plate, reads and validates its text, finds no recent duplicate, sends it to the backend, and logs the returned status.
- *Alternate flows*: no plate found (skip); OCR not confident (skip); text malformed (skip); duplicate within cooldown (skip); backend unreachable (log and continue); stream lost (retry).
- *Postcondition*: at most one report per distinct plate per cooldown window.

UC-2: Police officer is alerted (shared use case). The backend receives a *STOLEN* match from UC-1 and broadcasts it; the officer sees it on the dashboard. The node's part ends at the HTTP response.

User story. *As a camera-node operator, I want a stationary vehicle to be reported once, not on every frame, so that officers are not flooded with duplicate alerts.*

4.6 System / Solution Architecture

The architecture is in Figure 3.1. The detection-to-alert flow, which is the system's core value path, is below. The node performs the first step only.



The **integration contract** between the node and the backend is one endpoint:

```

POST /alerts/check-plate
Body:    { "license_plate": "BAA1234", "camera_id": "PHONE-NODE-1",
           "confidence_score": 92.4, "location_spotted": "..."}
Response: { "status": "STOLEN", "alert_id": 17, "vehicle": {...}}
           or { "status": "CLEAR" }
  
```

`confidence_score` is the detector's confidence as a percentage (0–100). The node holds no database access and no authentication of its own, and keeps only an in-memory list of recent readings, so restarting it loses no data of record.

4.7 Database / Data Design

The camera node has no database. It produces the data that the backend stores in the alerts table (report_id, camera_id, confidence_score, location_spotted, detected_at); that schema was designed by the backend member and is documented in the backend member's report and in Appendix C. The data design specific to this component is the **dataset**:

- one class, zambia-number-plate-detection (nc: 1);
- YOLO label format, one text file per image, one row per plate: class id and normalised centre x, centre y, width and height;
- data.yaml giving the train, validation and test folders.

4.8 Interface / Interaction Design

The node has no web interface. Its two interfaces are:

1. **The operator's console and preview window.** An OpenCV window titled "SVDS Plate Node" shows each frame with a green box around the detected plate and the cleaned plate text above it. Pressing q quits. The console prints [PLATE] backend status: CLEAR|STOLEN for each report.
2. **The .env file**, whose keys are MODEL_PATH, PHONE_STREAM_URL, BACKEND_URL, CAMERA_ID, LOCATION_SPOTTED, DETECT_CONF, PROCESS_EVERY_N_FRAMES, RESEND_COOLDOWN_SECONDS, DEDUP_SIMILARITY_THRESHOLD and DEVICE.

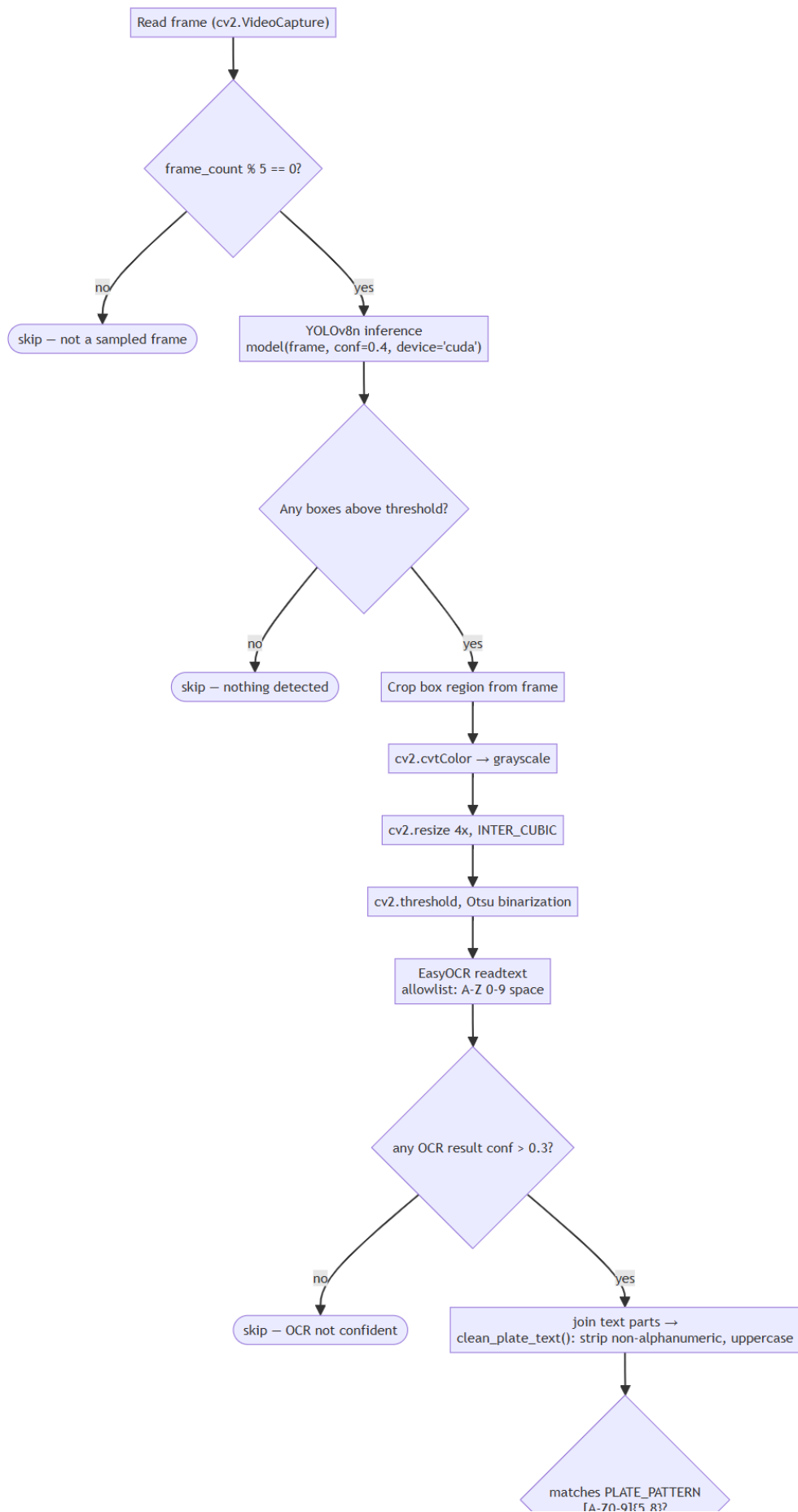
The web pages that display the node's output (Alerts, Cameras, Analytics) were designed by the frontend member.

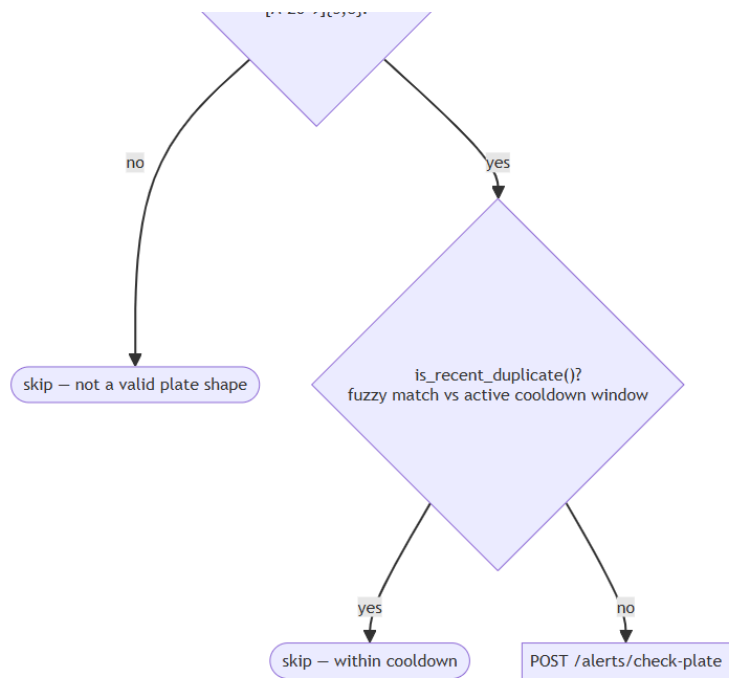
4.9 Algorithms / Models / Technical Design

4.9.1 Detector

A single-class YOLOv8n model fine-tuned from COCO-pretrained yolov8n.pt. It finds "where the plate is" only. Reading the characters is left to OCR so that the OCR engine can be replaced without retraining.

4.9.2 Read-and-report pipeline





Why each stage exists:

- **Frame skipping (every 5th frame):** running detection and OCR on every frame would fall behind the live stream on the available hardware.
- **Greyscale, 4× cubic upscale, Otsu threshold:** a raw crop is small and low-contrast; upscaling and binarising it before OCR is a standard way to make small text easier to read. (Its benefit was not measured in isolation in this project.)
- **Character allowlist:** stops the OCR engine from producing plausible-looking non-plate characters from background clutter.
- **OCR confidence floor (0.3):** discards low-confidence text fragments.
- **Regex validation ([A-Z0-9]{5,8}):** discards strings that cannot be a plate, rather than forwarding a guess.
- **Fuzzy cooldown:** see below.

4.9.3 Fuzzy duplicate suppression

A stationary vehicle is re-read on every processed frame, and OCR noise makes the string drift (for example ALX3665, ALK3665, AL3065 for one physical plate). A cooldown keyed on the *exact* string lets every variant through. The design instead keeps a list of (plate, timestamp) readings, drops entries older than the cooldown (default 15 s), and treats a new reading as a duplicate if its SequenceMatcher similarity to any remaining entry is at least a threshold (default 0.75).

The threshold is a trade-off. Too low, and two genuinely different plates that share most characters would be merged and one would go unreported. Too high, and OCR jitter would slip through as separate reports. The value 0.75 was chosen so that single-character OCR errors are

merged while clearly different plates are not (see tests in §6.3). It was not tuned on a labelled set of live readings, and that is a limitation.

4.10 Chapter Summary

The camera node is specified by ten functional requirements and six non-functional constraints. Its design is a staged pipeline (sample, detect, crop, pre-process, OCR, validate, de-duplicate, report) around a single-class YOLOv8n detector. It is connected to the rest of SVDS by a single HTTP contract.

Chapter 5: Implementation / Development

5.1 Introduction

This chapter describes how the AI/vision component was built: the dataset and model, and the camera node that runs them. It also summarises how the component was integrated with the other members' work, and identifies the student's own contribution.

5.2 Development Environment

Item	Detail
Machine	Windows 11 PC
GPU	NVIDIA Quadro P2000 (used for training and real-time inference)
Python environment	Conda environment <code>yo1ov8-env</code> with GPU-enabled PyTorch: Python 3.12.13, PyTorch 2.5.1 (CUDA 12.1 build), Ultralytics 8.4.83, EasyOCR 1.7.2, OpenCV 5.0.0
Camera source	Android phone running an IP Webcam app, streaming MJPEG over Wi-Fi or a phone hotspot
Annotation	Roboflow (browser)
Editor	Visual Studio Code
Version control	Git
Tests	pytest

5.3 System Components / Modules

Component	Location	Purpose	Built by
Dataset and <code>data.yaml</code>	<code>zambia-number-plate-detection-3/</code> (Roboflow export)	Labelled plate images	AI/vision (this report)
Trained detector	<code>runs/detect/svds-plate-detector-final-2/weights/best.pt</code>	Plate localisation	AI/vision (this report)
Camera node	<code>vrrs-node/plate_node.py</code>	Stream, detect, OCR, validate, de-duplicate, report	AI/vision (this report)
Plate utilities	<code>vrrs-node/plate_utils.py</code>	Plate cleaning and fuzzy de-duplication (pure Python)	AI/vision (this report)

Component	Location	Purpose	Built by
Node unit tests	vrrs-node/tests/test_plate_utils.py	8 pytest tests	AI/vision (this report)
check-plate endpoint, database, auth	vrrs-backend/	Matching, storage, access control	Backend and security member
WebSocket alerts, camera status and feed proxy	vrrs-backend/app/routers/alerts.py, system.py	Real-time delivery	Real-time and integration member
Web portals and analytics	vrrs-frontend/	User interface	Frontend member

5.4 Key Implementation Details

5.4.1 Dataset preparation and training

The dataset was annotated and exported from Roboflow in YOLOv8 format (§3.3). Training used the Ultralytics API with the settings recorded in the run's `args.yaml`:

Parameter	Value
Base weights	yolov8n.pt (COCO-pretrained)
Epochs	100 (patience 100, so no early stop)
Batch size	8
Image size	640 × 640
Device	GPU 0
Optimiser	auto (Ultralytics default selection)
Learning rate	1r0 = 0.01, final ratio 1rf = 0.01, cosine schedule off
In-training augmentation	mosaic 1.0, horizontal flip 0.5, HSV jitter, random erasing 0.4

The run `svds-plate-detector-final-2` is one of about twenty runs in `runs/detect/`. It is the one whose `best.pt` the node loads, via `MODEL_PATH`. This report does not attempt to reconstruct what differed between the other runs, because their settings were not recorded here.

5.4.2 The camera node (`plate_node.py`)

At start-up the node loads the detector once (`YOLO(MODEL_PATH)`, moved to the configured device) and creates one EasyOCR reader with GPU use following the device setting. It then opens the phone stream with `cv2.VideoCapture` and loops:

```

frame_count += 1
if frame_count % PROCESS_EVERY_N_FRAMES == 0:
    for res in model(frame, conf=DETECT_CONF, device=DEVICE, verbose=False):
        for box in res.bboxes:
            x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
            crop = frame[max(0, y1):y2, max(0, x1):x2]
            gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
            resized = cv2.resize(gray, None, fx=4, fy=4,
interpolation=cv2.INTER_CUBIC)
            thresh = cv2.threshold(resized, 0, 255,
                                cv2.THRESH_BINARY + cv2.THRESH_OTSU)[1]
            ocr = reader.readtext(thresh,
                                allowlist="ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 ")
            parts = [t.strip() for _, t, c in ocr if c > 0.3]
            plate = clean_plate_text(" ".join(parts)) if parts else None
            ...
            if plate and not is_recent_duplicate(recent_sightings, plate, now,
COOLDOWN):
                recent_sightings.append((plate, now))
                report_plate(plate, round(det_conf * 100, 1))

```

Defaults, all overridable in .env: detector confidence 0.4, every 5th frame, cooldown 15 s, similarity 0.75, device cuda.

report_plate posts the JSON payload of §4.6 with a 5-second timeout. A network or HTTP error is caught and printed, so a backend outage does not stop detection.

5.4.3 Plate cleaning and de-duplication (plate_utils.py)

These two functions were moved into their own module, free of cv2, torch, ultralytics and easyocr imports, so they can be unit tested without the GPU stack.

```

PLATE_PATTERN = re.compile(r"[A-Z0-9]{5,8}")

def clean_plate_text(raw: str) -> str | None:
    text = re.sub(r"^[A-Z0-9]", "", raw.upper())
    return text if PLATE_PATTERN.fullmatch(text) else None

def is_recent_duplicate(recent_sightings, plate, now, cooldown_seconds,
                        similarity_threshold=DEDUP_SIMILARITY_THRESHOLD):
    recent_sightings[:] = [(p, t) for p, t in recent_sightings
                           if now - t <= cooldown_seconds]
    return any(difflib.SequenceMatcher(None, plate, p).ratio() >=
similarity_threshold
               for p, _ in recent_sightings)

```

5.5 Integration

The node integrates with the rest of SVDS through one contract only (§4.6). Nothing in `vrrs-node` imports from `vrrs-backend`; the backend URL is configuration. The backend's `check-plate` endpoint, written by the backend member, normalises the received plate (strips non-alphanumeric characters, upper-cases it) and looks for an active report with that plate. This means the backend tolerates spacing differences the node might send. The WebSocket broadcast to police dashboards and the live-feed proxy that the Cameras page uses are the real-time and integration member's work. The node's responsibility ends when it receives the HTTP response.

The backend also contains a generic `POST /model/predict-image` endpoint. It is a placeholder for a server-hosted model that was never wired up, and it is **not** the detection path: inference runs in the node process. This was a deliberate design choice, so that the backend does not need a GPU or the computer-vision dependencies.

5.6 Challenges and Solutions

Challenge	Cause	Solution
Node process aborted on start-up on Windows	PyTorch and OpenCV each bundle Intel's OpenMP runtime (<code>libiomp5md.dll</code>); loading both aborts the process	<code>os.environ.setdefault("KMP_DUPLICATE_LIB_OK", "TRUE")</code> is set before either library is imported
Same stationary plate reported again and again	OCR output drifts between frames, so an exact-string cooldown never matched	Replaced with fuzzy matching (<code>SequenceMatcher</code> , threshold 0.75) over a list of recent readings; extracted to <code>plate_utils.py</code> and unit-tested
<code>import torch</code> failed with <code>WinError 126</code> on the development machine	Corrupted Visual C++ runtime files in System32	Repaired the "Microsoft Visual C++ 2022 X64 Minimum Runtime" package
<code>cv2.imshow</code> unavailable in the GPU environment	Both <code>opencv-python</code> and <code>opencv-python-headless</code> were installed, which corrupts the shared <code>cv2</code> package	Removed <code>opencv-python-headless</code> from <code>yolov8-env</code>
Node exits if the phone	<code>cv2.VideoCapture</code> fails to open, and the	Kept deliberately, so a misconfigured <code>PHONE_STREAM_URL</code> is noticed immediately. Once running, a dropped stream is

Challenge	Cause	Solution
stream is unreachable at start-up	node raises a clear <code>RuntimeError</code> naming the likely causes	retried in the loop
Phone IP address changes when it reconnects to the hotspot	DHCP	The operator updates <code>PHONE_STREAM_URL</code> in <code>.env</code> (a static IP or a stream-discovery step is future work)
Missed plates (recall 0.75, §6.4)	Motion blur, angle, small plates	Accepted as a trade-off for high precision. Sampling many frames per vehicle gives further chances to catch the plate

5.7 Individual Contribution (group project)

The student was responsible for the **AI/vision component** (parts 1 and 2 of the group's seven work areas, see Appendix A):

- collecting and annotating the plate dataset in Roboflow and exporting it in YOLOv8 format;
- training and evaluating the YOLOv8n plate detector, and selecting the weights the node uses;
- designing and writing the camera node `plate_node.py`: stream handling, frame sampling, detection, crop pre-processing, OCR and reporting;
- designing and writing `plate_utils.py` (plate validation and fuzzy de-duplication) and its eight unit tests;
- configuring the GPU environment for the node.

Work by other members that this component depends on, and which is **not** claimed here: the backend, database and check-plate matching logic; JWT authentication and role enforcement; the WebSocket alert broadcast, camera status and live-feed proxy; the React portals and analytics.

5.8 Chapter Summary

The AI/vision component is a fine-tuned YOLOv8n detector plus a camera node that samples a phone stream, crops and pre-processes detected plates, reads them with EasyOCR, validates and de-duplicates the text, and reports over one HTTP contract. Its main implementation problems were library conflicts on Windows and OCR noise, both solved in code.

Chapter 6: Testing, Results and Evaluation

6.1 Introduction

This chapter evaluates the AI/vision component. It reports what was measured, what was observed, and, just as importantly, what was **not** measured. Every number below comes from a file or a command output that can be inspected: `results.csv`, `eval_results.json`, the `pytest` output, and the node's own log.

6.2 Testing Strategy

Level	Technique	Target
Model evaluation	Precision, recall, mAP on held-out splits	The YOLOv8n plate detector
Unit testing	pytest on pure functions, with no GPU stack	<code>plate_utils.py</code> (cleaning, de-duplication)
Robustness observation	Running the node against unreachable and dropped streams	<code>plate_node.py</code> start-up and reconnection
System testing	Live phone stream → node → backend → dashboard	Partly achieved: live readings reached the backend; no STOLEN alert recorded (§6.4.3)
Acceptance / user testing	None carried out	See §6.5

6.3 Test Cases and Results

Table 6.1: Test cases for the AI/vision component

ID	Test Case	Expected Result	Status
TC-01	<code>clean_plate_text</code> given lower case and punctuation ("baa 1234", "BAA-1234")	"BAA1234" in both cases	Pass
TC-02	<code>clean_plate_text</code> given text shorter than 5 or longer than 8 characters	None (discarded)	Pass
TC-03	<code>clean_plate_text</code> given only symbols	None	Pass
TC-04	Same plate re-read inside the cooldown window	Treated as duplicate	Pass

ID	Test Case	Expected Result	Status
TC-05	OCR-jitter variants of one plate (ALX3665, ALK3665, AL3065-style) inside the window	Treated as duplicates	Pass
TC-06	A genuinely different plate inside the window	Not a duplicate	Pass
TC-07	Same plate after the cooldown has expired	Not a duplicate	Pass
TC-08	Expired entries in the sightings list	Pruned from the list	Pass
TC-09	Detector on the validation split (407 images)	Precision above 0.95 and mAP50 above 0.75	Pass (0.990 / 0.797)
TC-10	Detector on the test split (7 images)	Reported, no threshold set	Reported (§6.4.2)
TC-11	Start node with an unreachable stream	Clear error, not a silent hang	Pass (observed, §6.4.3)
TC-12	Stream drops while the node is running	Node retries instead of exiting	Pass (observed in the node's log, §6.4.3)
TC-13a	Live phone stream → node → backend, plate not on the stolen list	Backend returns CLEAR	Pass (8 live readings, §6.4.3)
TC-13b	Live stream → node → backend → dashboard alert for an active report	STOLEN and a dashboard alert	Not verified
TC-14	End-to-end OCR accuracy on real plates	A measured character or plate accuracy	Not measured (no text ground truth)

TC-01 to TC-08 correspond one-to-one to the eight tests in `vrrs-node/tests/test_plate_utils.py`. The suite was re-run while preparing this report: **8 passed in 0.06 s** (Appendix D). The exact inputs are in the test file; TC-04 to TC-08 above are summaries of them.

6.4 Experimental Results / Performance Results

6.4.1 Training results (validation split)

Final-epoch (epoch 100 of 100) values from `results.csv`, alongside epoch 97, which Ultralytics' fitness measure ($0.1 \times \text{mAP50} + 0.9 \times \text{mAP50-95}$) selects as best:

Metric	Epoch 100	Best-fitness epoch (97)
Precision	0.988	0.990

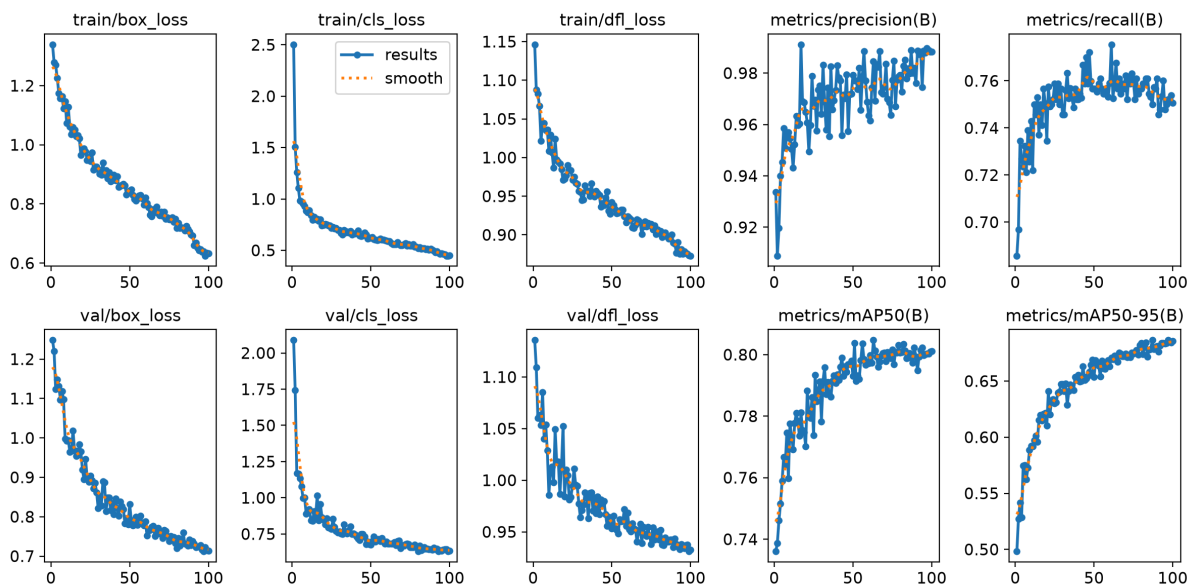
Metric	Epoch 100	Best-fitness epoch (97)
Recall	0.751	0.752
mAP50	0.801	0.800
mAP50-95	0.686	0.686

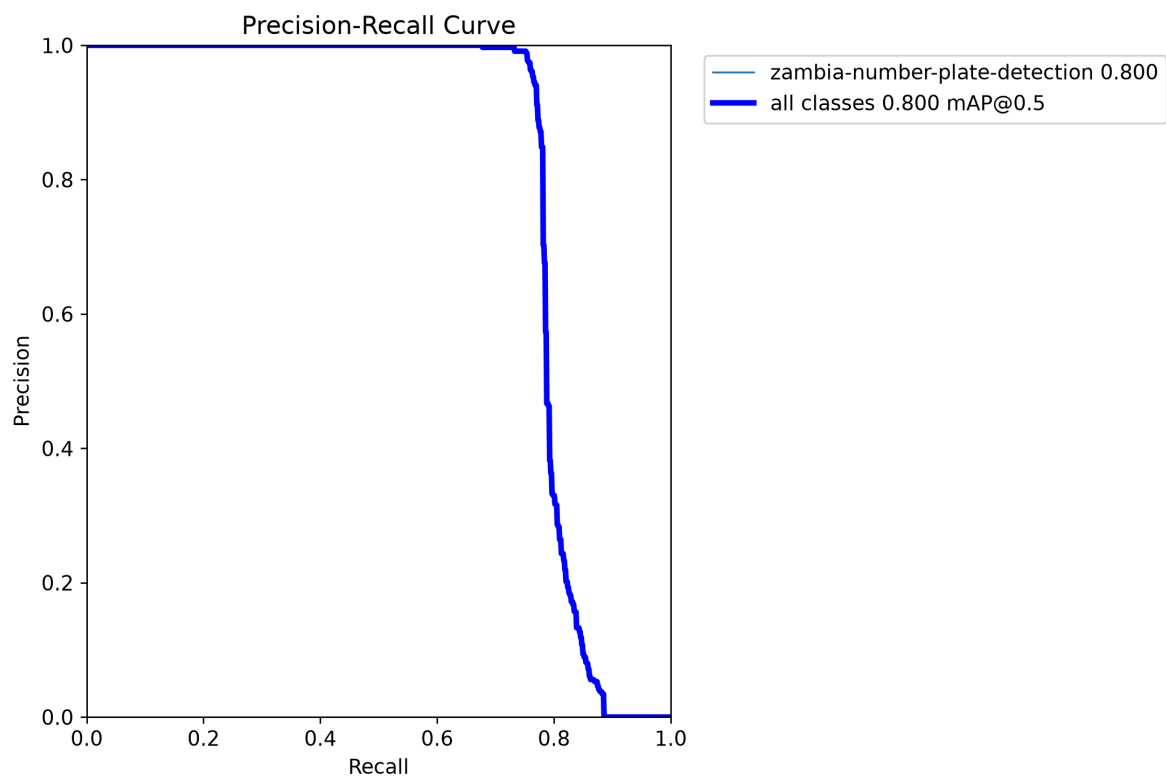
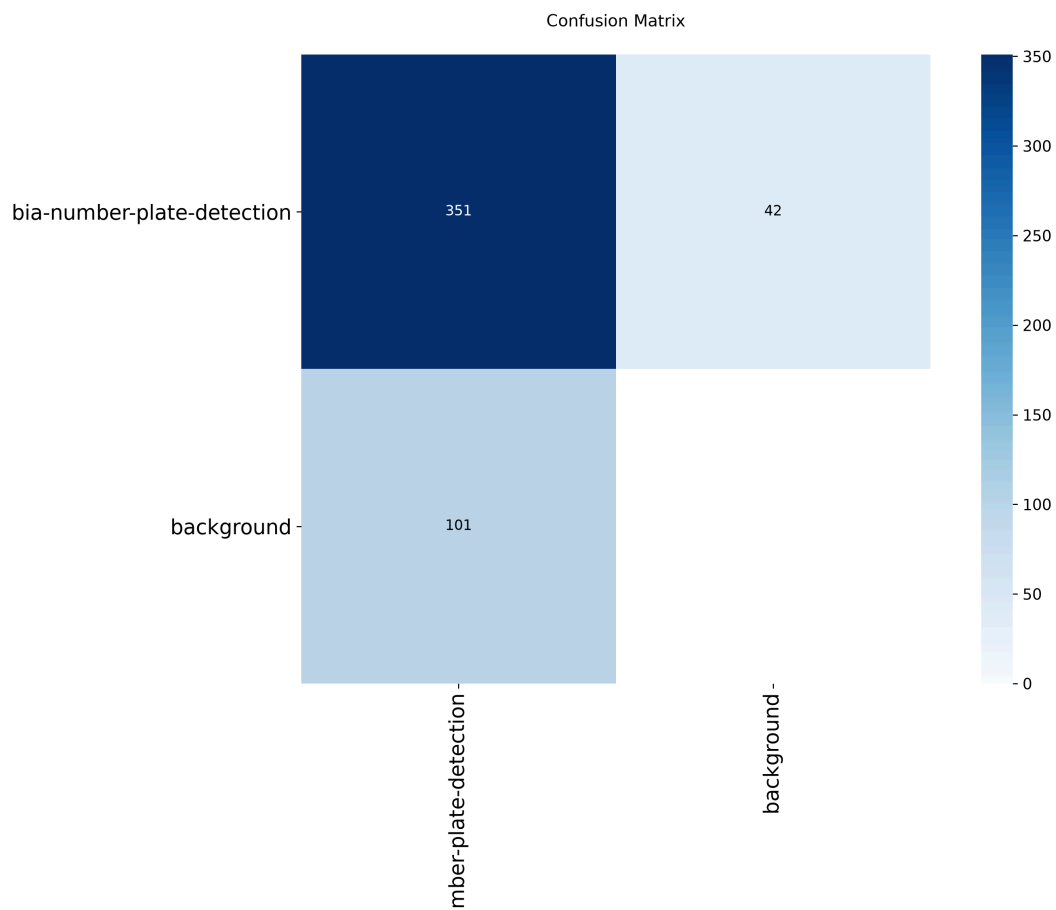
best.pt is the checkpoint saved at the best-fitness epoch, so the deployed weights correspond to the epoch-97 column (assuming training was not resumed, which the record does not show).

Re-evaluation of the deployed best.pt with evaluate_model.py (validation split, 407 images, 452 plate instances):

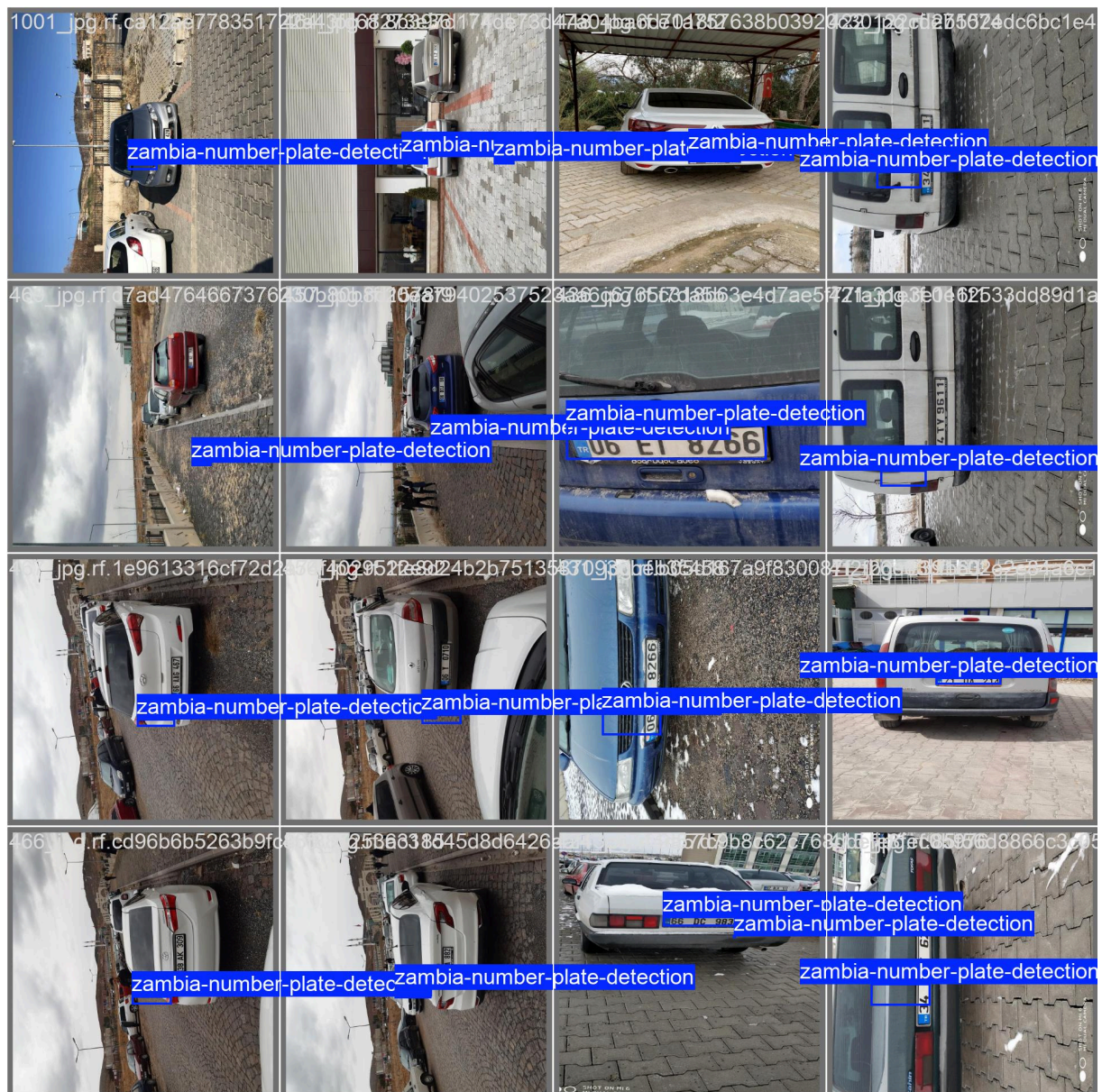
Split	Images	Instances	Precision	Recall	mAP50	mAP50-95
Validation	407	452	0.990	0.752	0.797	0.690
Test	7	11	0.764	0.909	0.881	0.725

Inference time on the Quadro P2000 was about 11.2 ms per image for the model alone (preprocess 0.8 ms, postprocess 1.9 ms). This excludes OCR, the HTTP call and video decoding, so it is **not** the frame rate of the whole node.









6.4.2 Test split

The test split holds only 7 images with 11 plates. Its precision (0.764) is far below the validation precision (0.990), and its recall (0.909) is above it. With 11 instances, one or two false detections move precision by tens of percentage points, so these figures show that the model works on unseen images but **cannot be used to estimate the true error rates**. A larger test set is needed.

6.4.3 Robustness observations

- **Live readings reached the backend (TC-13a).** The node's log `vrrs-node/node_live.log` (last modified 14 September 2026) records a real run: the detector loaded on cuda, the node connected to the phone stream, and eight readings were posted to the backend, each answered backend status: CLEAR. The readings were ALK3665, AK3665, ALX3665, ALK3665,

ALLX3665, ALX3665, XI5998 and G9QEXIH. The first six differ by one or two characters and appear to be repeated, noisy reads of one plate; the log does not say what the true plate was. This run shows the phone → node → backend path working, and that the backend was reached and answered.

- **The same log is the evidence for the de-duplication fix.** Six near-identical reports of one apparent plate in one run is the OCR-jitter problem described in §4.9.3. The run predates the fuzzy-matching fix (the fix and its unit tests were written afterwards), so it shows the problem, not the fix working on live footage.
- **Unreachable stream at start-up (TC-11).** When the system was later started, the phone's address was unreachable (a TCP test to the configured address failed). The node loaded YOLOv8n and EasyOCR, then stopped with an explicit `Could not open video stream...` error naming the likely causes, as designed.
- **Stream loss while running (TC-12).** The same live log ends with an `OpenCV/FFmpeg stream timeout` after about 30 s followed by `Lost connection to stream, retrying...` (twice), showing the retry branch executing instead of the node exiting. The log does not show a later successful reconnection, so recovery *after* the stream returns is not demonstrated.

6.4.4 What was not verified

- **No recorded STOLEN match** (TC-13b). All eight live readings returned `CLEAR`. There is no recorded run in which a real vehicle's plate matched an active report and produced an alert on the dashboard, and the camera-node preview window has not been captured. Latency from a plate becoming visible to an alert appearing has therefore **not been measured**, and no latency figure is claimed. It is also not known whether the plate in the log was ever on the stolen list.
- **No OCR accuracy figure** (TC-14). The dataset labels where plates are, not what they say, so read accuracy cannot be computed from it. Producing that number needs a set of plate crops with typed ground-truth strings.
- **No comparison of alternatives**, for example with and without the greyscale, upscale and threshold stage, or with other similarity thresholds.

6.5 User Evaluation

None carried out. No officers, operators or members of the public tested the node, and no survey or interview data exist.

6.6 Analysis of Results

- The validation metrics show a detector that is **precise but misses about a quarter of plates** in still images (precision 0.99, recall 0.75). In `results.csv`, mAP50 is already about 0.78 at epoch 10 and reaches about 0.80 at epoch 100, and mAP50-95 rises from about 0.59 to 0.686 with only about +0.004 over the last ten epochs. More epochs alone would therefore give little. More or more varied data, especially examples of the missed cases, is the more promising lever.
- The re-run of the deployed weights matches the recorded training metrics to within about 0.005 on every measure, which confirms that `best.pt` is the model that was trained and evaluated, and that the recorded numbers are reproducible.
- The test split contradicts the validation picture on precision. With 11 instances it is inconclusive, but it is a reminder that the validation score is measured on images from the same collection effort and may overstate performance on new scenes.
- The unit tests show that the parts of the pipeline that could be isolated behave as specified, in particular that jittery re-reads of one plate are suppressed while a different plate is not.
- The live log shows the pipeline reading real plates and reaching the backend, but the absence of OCR accuracy and of a recorded `STOLEN` alert means the *pipeline as a whole* is supported by design, component evidence and a partial live run, not by a measured end-to-end result.

6.7 Objective-by-Objective Evaluation

Objective	Evidence	Verdict
O1: Collect and annotate a dataset with train/validation/test splits	1,890 labelled images in three splits, no source-id overlap between splits (§3.3). Original photograph count unconfirmed; test split very small	Achieved , with the caveats stated
O2: Train a detector reaching precision above 0.95 and mAP50 above 0.75 on validation	0.990 precision, 0.797 mAP50 (re-run); 0.988 and 0.801 at epoch 100	Achieved
O3: Read plate text with OCR and validate its format	Implemented in <code>plate_node.py</code> and <code>clean_plate_text</code> ; validation logic unit-tested (TC-01 to TC-03). OCR read accuracy not measured	Achieved in implementation; accuracy unproven
O4: Suppress repeated reports despite OCR noise	Fuzzy de-duplication implemented; TC-04 to TC-08 pass	Achieved at unit level; not measured on live footage
O5: Decoupled camera node that reports over HTTP and recovers from stream loss	Separate process, single HTTP contract; 8 live readings posted and answered <code>CLEAR</code> (TC-13a); retry branch observed (TC-12); no	Largely achieved: reporting demonstrated live; alert path and recovery not demonstrated

Objective	Evidence	Verdict
	STOLEN alert recorded, and reconnection after recovery not shown	
O6: Evaluate the model and node quantitatively and by automated tests	Metrics in §6.4, 8 passing tests	Achieved for the model and the pure logic; not for the whole pipeline

6.8 Individual Results

All results in this chapter are attributable to the student's own component. Results of the backend (including check-plate tests), real-time delivery and frontend are in the other members' reports and are not reproduced or claimed here.

6.9 Chapter Summary

The detector meets its numeric targets on the validation split (precision 0.990, mAP50 0.797), the eight unit tests pass, and the node's start-up and stream-loss behaviour was observed. A live run showed eight readings reaching the backend, but a STOLEN alert on the dashboard, OCR accuracy and latency were not recorded, and the seven-image test split is too small to be conclusive.

Chapter 7: Discussion

7.1 Introduction

This chapter interprets the results of Chapter 6 in relation to the problem and the objectives, compares them with related work, states the contributions, and discusses limitations.

7.2 Interpretation of Findings

The detector suits the use case. Precision of about 0.99 means almost every box the model draws is a real plate. Recall of about 0.75 means roughly one plate in four is missed in a still image. For a system whose alerts go to police, this is the right side of the trade-off. A missed plate in one sampled frame is likely to be caught in a later frame of the same passing vehicle, whereas a false box would trigger OCR and possibly a wrong match. This reasoning is an argument from design, not a measurement: the per-vehicle detection rate across a video was not measured.

mAP50 versus mAP50-95. The gap between mAP50 (about 0.80) and mAP50-95 (about 0.69) shows that boxes are usually in the right place but not always tight. That matters little for this pipeline, because the crop is upscaled and binarised before OCR and a slightly loose box still contains the plate. A box that clips characters would hurt OCR more than one that is too large, and this was not analysed.

Software-level noise handling matters as much as the model. The largest practical problem on real footage was not detection but OCR jitter on a stationary vehicle. Fuzzy de-duplication fixed it in a few lines of code, and unit tests now pin the behaviour down.

Answer to research question 1 (can YOLOv8n plus an open-source OCR engine read Zambian plates well enough for real-time matching on a phone camera and consumer GPU?). The detection half is supported by measured metrics. The reading half is **not yet supported by measurement**: no labelled plate-text set exists, so end-to-end read accuracy is unknown, and no live run producing a STOLEN alert was recorded. The honest answer is "detection: yes, on this evidence; reading and real-time matching: plausible and implemented, but unproven".

Answer to research question 2 (can the pipeline be a decoupled component talking only over HTTP?). Yes. The node is a separate process with a single-endpoint contract, and it has been run and tested without importing anything from the backend.

7.3 Comparison with Existing Work

No accuracy figures from OpenALPR, Plate Recognizer or published ALPR studies were reproduced on the Zambian test set, so no numerical comparison is made. The comparison in §2.4 is on design properties: the project is self-hosted and open source, is trained on locally collected plates, and is linked to a reporting workflow. Reported mAP values from other datasets are not comparable with these results, because the datasets, the number of classes and the difficulty differ.

7.4 Contributions of the Project

- **Practical:** a working prototype in which a phone camera and a consumer GPU feed plate readings into a police-facing stolen-vehicle registry.
- **Technical:** a single-class YOLOv8n plate detector fine-tuned on locally collected Zambian plate images, released as reproducible artifacts (training arguments, per-epoch metrics, curves); a noise-tolerant read-validate-de-duplicate pipeline; a GPU-free unit-testable core.
- **Architectural:** a decoupled camera-node design that could be moved to fixed roadside cameras without backend changes.

7.5 Limitations and Implications

Limitation	Implication for interpreting the results
Small dataset with uncertain source-image count	The metrics may not generalise to other cameras, regions, plate styles or lighting
Splits checked for leakage by filename only	No source-image name appears in more than one split (verified, §3.3), so augmented copies of one photograph did not leak between training and validation. This rests on filenames being faithful to the original photographs
Test split has 7 images	Test-split metrics have wide uncertainty and cannot support strong claims
No plate-text ground truth	OCR accuracy, and therefore the true rate of correct end-to-end matches, is unknown
No recorded STOLEN alert from a live run	Latency and the full alert path are not established
Single machine and camera	Performance on other hardware is unknown; CPU-only operation falls behind the stream
Fuzzy threshold not tuned on live data	The value 0.75 could merge different plates that differ by one or two characters
Plate crops are pre-processed the same way in all lighting	Night-time and glare performance is unknown

7.6 Chapter Summary

The measured results support the detector's suitability for a precision-first police use case. Reading accuracy and live performance remain unmeasured, and the report states this instead of extrapolating. The main contribution is a decoupled, tested pipeline with a locally trained model.

Chapter 8: Conclusion and Recommendations

8.1 Introduction

This chapter summarises the project, states the conclusions the evidence supports, reviews the individual objectives, and gives recommendations and future work.

8.2 Summary of the Project

Stolen-vehicle recovery in Zambia relies on officers chancing to recognise a plate. The group built SVDS on top of the VRRS registry so that a plate seen by a camera is checked automatically against active reports. This report covered the AI/vision component. A single-class YOLOv8n detector was fine-tuned on 1,890 labelled images of Zambian plates, and a camera node was built around it. The node samples a phone video stream, crops and pre-processes detected plates, reads them with EasyOCR, validates the text, suppresses noisy duplicate reads by fuzzy matching, and reports each reading to the backend over one HTTP endpoint.

8.3 Conclusions

1. A small, locally collected dataset is enough to train a plate detector with high precision (0.990) and moderate recall (0.752) on the validation split, at about 11 ms per image on a Quadro P2000.
2. OCR noise, not detection, was the main practical problem on live footage. A fuzzy similarity check handles it, and unit tests confirm the behaviour.
3. A decoupled node that talks to the backend over one HTTP contract works: live readings from a phone stream reached the backend and were answered.
4. The evidence does not yet show end-to-end read accuracy, alert latency, or a recorded STOLEN alert from a live run. The test split (7 images) is too small to support strong claims.

8.4 Achievement of Objectives

Objective	Status
O1 Dataset	Achieved (original photograph count to be confirmed)
O2 Detector above 0.95 precision and 0.75 mAP50	Achieved (0.990 / 0.797)
O3 OCR and validation	Achieved in implementation; read accuracy unmeasured
O4 Duplicate suppression	Achieved at unit level; not measured on live footage

Objective	Status
O5 Decoupled node	Largely achieved; live reporting shown, alert path and reconnection not shown
O6 Evaluation	Achieved for the model and pure logic, not the whole pipeline

Research question 1 is answered for detection and left open for reading accuracy. Research question 2 is answered yes.

8.5 Recommendations

- Before any field use, build a labelled set of plate crops with typed text and measure OCR accuracy.
- Enlarge the test split and include night, glare, motion blur and angled shots.
- Put `/alerts/check-plate` behind a private network or an API key.
- Keep a human officer in the loop: treat every alert as a lead, never as proof.
- Confirm the licences of YOLOv8 (AGPL-3.0), EasyOCR and the dataset before any deployment beyond a prototype.

8.6 Future Work

- Record a full live run that produces a `STOLEN` alert, and measure latency.
- Tune the fuzzy threshold on labelled live readings.
- Evaluate the pre-processing stage and alternative OCR engines.
- Read GPS location from the phone instead of a configured string.
- Export the model to ONNX or TensorRT to reduce GPU dependence, and support several camera nodes.
- Discover the phone stream address automatically instead of editing `.env`.

References

1. Ultralytics. "YOLOv8 Documentation." <https://docs.ultralytics.com> (accessed 2026).
2. JaidevAI. "EasyOCR." <https://github.com/JaidevAI/EasyOCR> (accessed 2026).
3. Roboflow. "zambia-number-plate-detection, version 3" (workspace nicholass-workspace-j3yig), CC BY 4.0. <https://universe.roboflow.com/nicholass-workspace-j3yig/zambia-number-plate-detection/dataset/3>
4. OpenALPR. "OpenALPR: Open Source Automatic License Plate Recognition." <https://github.com/openalpr/openalpr>
5. Plate Recognizer. "Plate Recognizer ALPR." <https://platerecognizer.com>
6. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. <https://opencv.org>
7. Otsu, N. (1979). A threshold selection method from gray-level histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62-66.
8. Redmon, J., Divvala, S., Girshick, R., Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016.
9. Python Software Foundation. "difflib: Helpers for computing deltas." <https://docs.python.org/3/library/difflib.html>
10. FastAPI. "FastAPI Documentation." <https://fastapi.tiangolo.com>
11. pytest. "pytest Documentation." <https://docs.pytest.org>
12. Sommerville, I. (2011). Software Engineering, 9th ed. Pearson.

[Add any interviews, Zambia Police Service correspondence or supervisor notes you used, and format all entries to the department's required style.]

Appendix A: Individual Contribution Statement (Group Projects)

Purpose of this Statement

This statement identifies the work for which the student was primarily responsible. It is consistent with the main report. Work by other group members is acknowledged and is not claimed.

A.1 Project and Group Information

Item	Details
Project Title	SVDS: A Camera-Based Stolen Vehicle Detection System for the Zambia Police Service (built on the VRRS registry)
Student Name	Moomba Nicholas Katapazi
Computer Number	[COMPUTER NUMBER]
Supervisor	[SUPERVISOR NAME]
Group Size	4
Group Members	1. Moomba Nicholas Katapazi, [COMPUTER NUMBER] (AI/vision) 2. [NAME], [COMPUTER NUMBER] (Backend and security) 3. [NAME], [COMPUTER NUMBER] (Real-time and integration) 4. [NAME], [COMPUTER NUMBER] (Frontend, analytics, testing and documentation)

A.2 Overall Group Project

The group built **SVDS/VRRS**, a system with which the public files stolen-vehicle reports and police officers are alerted automatically when a camera sees a reported vehicle. It has three independently running processes: a FastAPI and PostgreSQL backend, a React frontend with three role-based portals (Reportee, Police, Admin), and a camera node that reads number plates from a phone's video stream using a custom-trained YOLOv8 detector and OCR. The common problem is that stolen-vehicle recovery depends on officers chancing to recognise a plate. The system's core flow is: camera node reads a plate, the backend matches it against active reports, and connected police dashboards receive an alert over WebSocket.

A.3 Individual Role and Responsibilities

The group's work was divided into four roles. The student's role was **Role 1: AI/vision**, which covers work areas 1 (detection model) and 2 (camera node).

No.	Area of Responsibility	Description
1	Detection model (AI)	Collect and label Zambian plate images, train YOLOv8n to find the plate, evaluate it
2	Camera node (edge)	Read the phone video stream, run detection and OCR, clean and validate the text, drop duplicate reads, send results to the backend

Other roles, for reference: **Role 2** backend API, database, check-plate logic, JWT, bcrypt, role enforcement, user management and audit log; **Role 3** WebSocket alerts, camera status and live-feed proxy, system health, and the phone-to-server wiring; **Role 4** the React portals, analytics, integration tests, run scripts and documentation.

A.4 Detailed Individual Contribution

- **Data collection and annotation:** collected Zambian plate images with a phone camera; annotated them in Roboflow with one class; exported in YOLOv8 format with the recorded pre-processing and augmentation.
- **Model development:** trained YOLOv8n from COCO-pretrained weights on GPU; compared runs; selected the weights used by the node (svds-plate-detector-final-2/weights/best.pt).
- **Algorithm development:** designed the read-and-report pipeline (frame sampling, crop, greyscale, 4× upscale, Otsu threshold, EasyOCR with allowlist, confidence floor, regex validation) and the fuzzy duplicate-suppression algorithm.
- **Software development:** wrote vrrs-node/plate_node.py and vrrs-node/plate_utils.py.
- **Testing:** wrote the eight unit tests for the node's pure logic and evaluated the detector.
- **Deployment and configuration:** set up the GPU Python environment and resolved the OpenMP, Visual C++ runtime and OpenCV package conflicts.
- **Documentation:** this report's AI/vision material, including the detection chapter content and the node's .env documentation.

A.5 Individual Deliverables

No.	Deliverable	Student's Contribution
1	Annotated dataset (zambia-number-plate-detection, v3)	Collected, annotated, exported

No.	Deliverable	Student's Contribution
2	Trained detector <code>best.pt</code> and its training record (<code>args.yaml</code> , <code>results.csv</code> , <code>curves</code>)	Trained, evaluated, selected
3	Camera node <code>plate_node.py</code>	Designed and implemented
4	<code>plate_utils.py</code> with 8 unit tests	Designed, implemented, tested
5	Node configuration (<code>.env</code> keys) and set-up instructions	Defined and documented

A.6 Contribution to the Main System or Research Output

The student produced the component that turns camera video into plate strings. It is the input to the backend's `POST /alerts/check-plate` endpoint. The endpoint itself, the matching against reports and the storage of alerts were built by the backend member. The alert broadcast and camera status display were built by the real-time and integration member. The pages that show the results were built by the frontend member. The `check-plate` contract (§4.6) was agreed jointly between the node and the backend.

A.7 Individual Implementation and Technical Work

Modules and components implemented by the student: the plate dataset and `data.yaml`; the trained YOLOv8n detector; `plate_node.py` (stream capture and reconnection, frame sampling, detector inference, crop pre-processing, OCR call, reporting to the backend, preview window); `plate_utils.py` (`clean_plate_text`, `is_recent_duplicate`); `tests/test_plate_utils.py`. Datasets and experiments handled: the Roboflow export (1,890 images), the 100-epoch training run, and the validation and test-split evaluations in Chapter 6.

A.8 Individual Testing and Evaluation Contribution

The student designed and ran the node unit tests (8 of 8 passing) and the detector evaluations reported in Chapter 6, and wrote the analysis in Chapters 6 and 7. The backend pytest suite and the manual browser checks of the web application were carried out by other members and are not claimed here.

A.9 Collaboration with Other Group Members

- The node depends on the **backend member's** `check-plate` endpoint: its request format and its `CLEAR/STOLEN` response were agreed jointly. The backend's plate normalisation

means the node does not have to reproduce the backend's matching rules.

- The **real-time and integration member's** alert broadcast and camera status/feed proxy depend on the node's CAMERA_ID and on the phone stream address, which both share.
- The **frontend member's** pages display the node's detections and status.

A.10 Contribution Summary

1. **Work primarily completed by the student:** the dataset, the trained detector, the camera node, the plate utilities and their tests.
2. **Work completed jointly with other members:** the node-to-backend contract; shared configuration of the phone stream address; end-to-end start-up through run-all.ps1.
3. **Work primarily completed by other members and used here:** the backend, database and check-plate matching; authentication and access control; the WebSocket alert delivery; the web portals.

A.11 Estimated Contribution

The percentages below are the student's own estimate and must be completed by the student. They were not derived from any record.

Project Activity	Individual Contribution	Collaborative Contribution
Problem Analysis	[%]	[%]
Literature Review	[%]	[%]
Requirements Analysis	[%]	[%]
System/Research Design	[%]	[%]
Implementation / Development	[%]	[%]
Testing and Evaluation	[%]	[%]
Documentation	[%]	[%]
Other	[%]	[%]

A.12 Statement of Authorship and Contribution

I confirm that the contribution described in this statement accurately represents the work that I personally undertook as part of the group project. I have identified collaborative work and have not knowingly claimed the work of another group member as my own.

Student Name: Moomba Nicholas Katapazi **Computer Number:** ____ **Signature:** ____ **Date:** ____

A.13 Supervisor Verification

I confirm that, to the best of my knowledge, the contribution described above is consistent with the student's participation in the group project.

Supervisor Name: ____ ***Signature:*** ____ **Date:** ____

Appendix B: User Manual / Installation Guide

This guide covers the camera node (the student's component). Set-up of the backend and frontend is documented in the project `README.md` and in the other members' reports.

B.1 Requirements

- Windows 11 (or another OS with a CUDA-capable GPU setup).
- A Python environment with GPU-enabled `torch`, `ultralytics`, `easyocr`, `opencv-python` (not the headless build, since the preview window needs GUI support), `requests`, `python-dotenv`. The development environment is listed in §5.2.
- The trained weights `best.pt`.
- An Android phone running an IP Webcam app, on the same network or hotspot as the PC.
- The SVDS backend running and reachable.

B.2 Configure

```
cd vrrs-node
cp .env.example .env
```

Edit `.env`:

Key	Meaning	Default
PHONE_STREAM_URL	Stream address shown by the phone app, with <code>/video</code> appended	<code>http://192.168.1.100:8080/video</code> (example)
BACKEND_URL	Base URL of the SVDS backend	<code>http://127.0.0.1:8000</code>
CAMERA_ID	Identifier sent with every reading	<code>PHONE-NODE-1</code>
LOCATION_SPOTTED	Location text sent with every reading	Unknown
MODEL_PATH	Path to <code>best.pt</code>	see <code>.env.example</code>
DETECT_CONF	Detector confidence threshold	0.4
PROCESS_EVERY_N_FRAMES	Run detection on every Nth frame	5

Key	Meaning	Default
RESEND_COOLDOWN_SECONDS	Duplicate-suppression window	15
DEDUP_SIMILARITY_THRESHOLD	Fuzzy match threshold, 0–1	0.75
DEVICE	cuda or cpu	cuda

The phone's address changes when it reconnects, so update `PHONE_STREAM_URL` each session.

B.3 Run

1. Start the IP Webcam app on the phone and tap **Start server**.
2. Start the backend.
3. From `vrrs-node`, run `python plate_node.py` in the GPU environment (or run `run-all.ps1` from the project root to start all three components).
4. Wait about 20–30 seconds while the models load.
5. A window titled "SVDS Plate Node" opens. A green box and the plate text appear on detected plates. The console prints `[PLATE] backend status: CLEAR` or `STOLEN`.
6. Press `q` in the preview window to quit.

B.4 Troubleshooting

Symptom	Likely cause	Action
Could not open video stream at start-up	Wrong or unreachable <code>PHONE_STREAM_URL</code> ; phone app not running; different network	Check the phone app, network and <code>.env</code>
Lost connection to stream, retrying... repeated	Phone went to sleep or left the network	Wake the phone, check the network; the node retries automatically
failed to reach backend	Backend not running or wrong <code>BACKEND_URL</code>	Start the backend; detection continues meanwhile
Crash on start mentioning <code>libiomp5md.dll</code>	Duplicate OpenMP runtimes	The node sets <code>KMP_DUPLICATE_LIB_OK</code> ; make sure it is running the unmodified <code>plate_node.py</code>
<code>import torch</code> fails with <code>WinError 126</code>	Damaged Visual C++ runtime	Repair "Microsoft Visual C++ 2022 X64 Minimum Runtime"
Preview window fails to open	Headless OpenCV installed alongside the normal build	Uninstall <code>opencv-python-headless</code>

B.5 Run the unit tests

```
cd vrrs-node  
python -m pytest tests/ -v
```

The tests need only pytest; they do not import torch, cv2 or easyocr.

Appendix C: Additional System Designs / Diagrams

C.1 Database schema (designed by the backend member)

The node's readings end up in the alerts table. The full schema of the four tables is shown for reference.

USERS			
int	id	PK	
string	first_name		
string	last_name		
string	email	UK	
string	phone		
string	password_hash		
string	role		reportee police admin
string	national_id		
string	badge_number		
bool	is_active		
datetime	created_at		

files (reported_by)

REPORTS			
int	id	PK	
int	reported_by	FK	
string	owner_name		
string	license_plate		
string	vehicle_make		
string	vehicle_model		
string	vehicle_color		
string	vehicle_year		
string	chassis_number		
datetime	incident_date		
string	status		under_review missing found
text	last_seen_location		
text	description		
datetime	report_date		
datetime	resolved_date		
int	resolved_by	FK	

triggers (report_id)

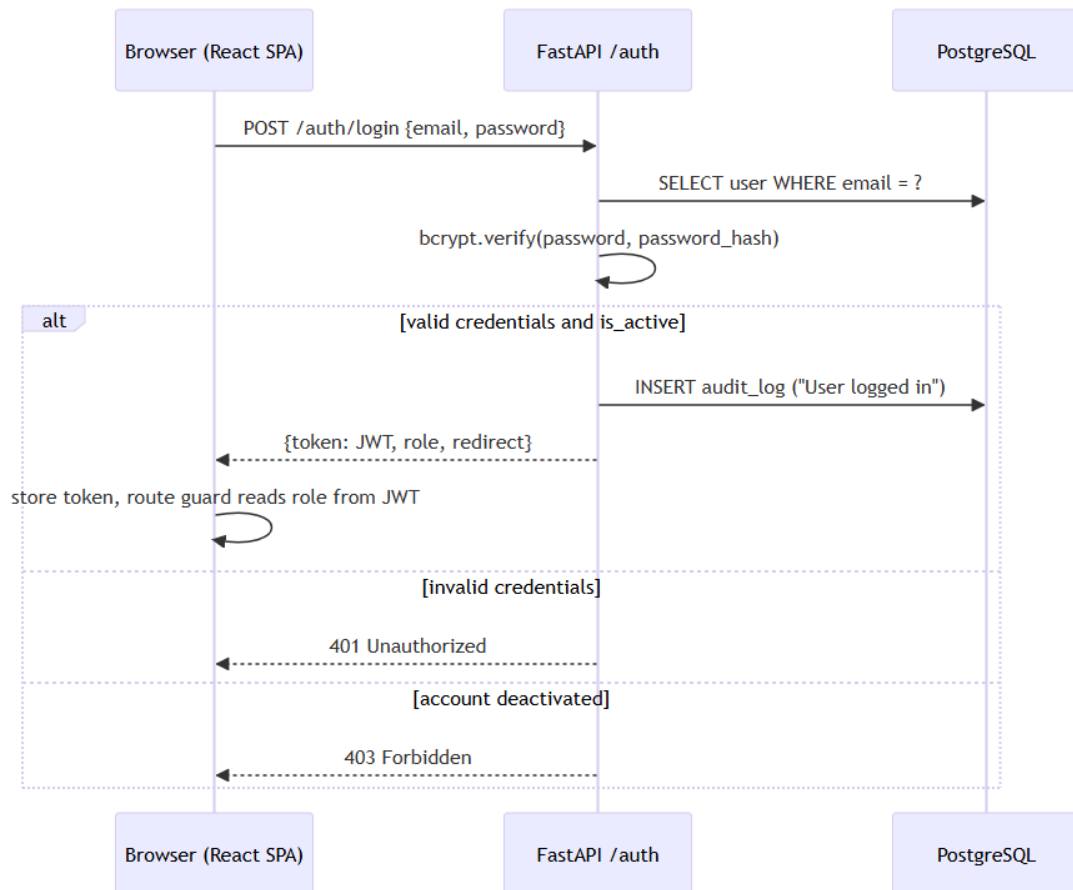
ALERTS			
int	id	PK	
int	report_id	FK	
text	location_spotted		
string	camera_id		

performs (performed_by)

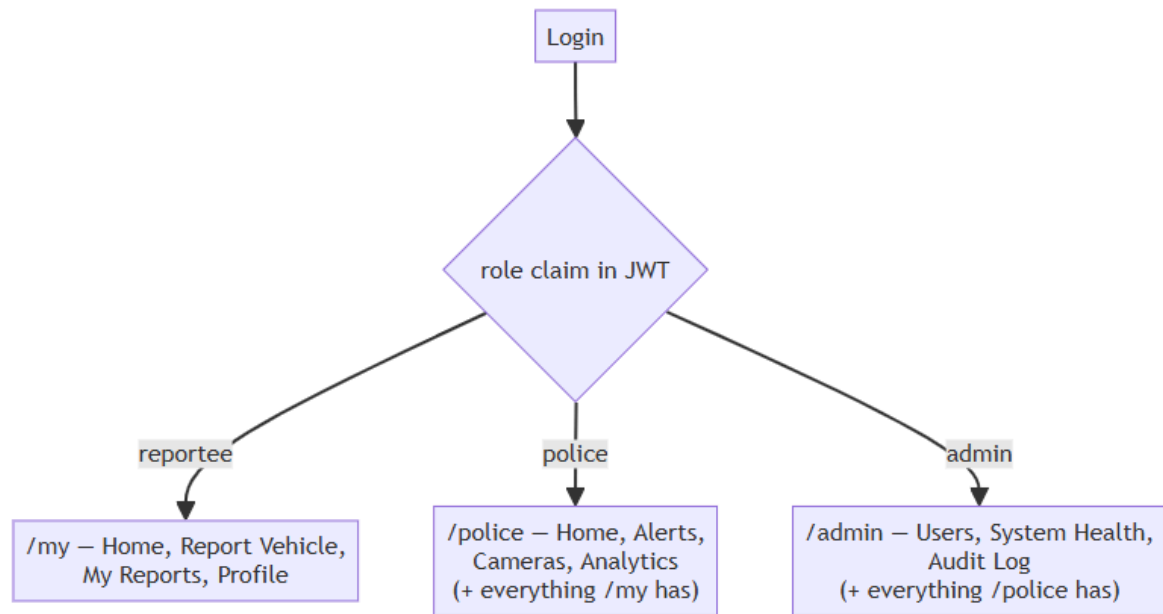
AUDIT_LOG			
int	id	PK	
int	performed_by	FK	
string	action		
string	target_table		
int	target_id		
datetime	timestamp		

float	confidence_score	
string	image_path	
bool	is_read	
datetime	detected_at	

C.2 Authentication sequence (backend member)



C.3 Role-based route access (backend and frontend members)



Appendix D: Additional Test Cases and Results

D.1 Raw output of the node unit tests

```
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0
rootdir: ...\vrrs-node
collected 8 items

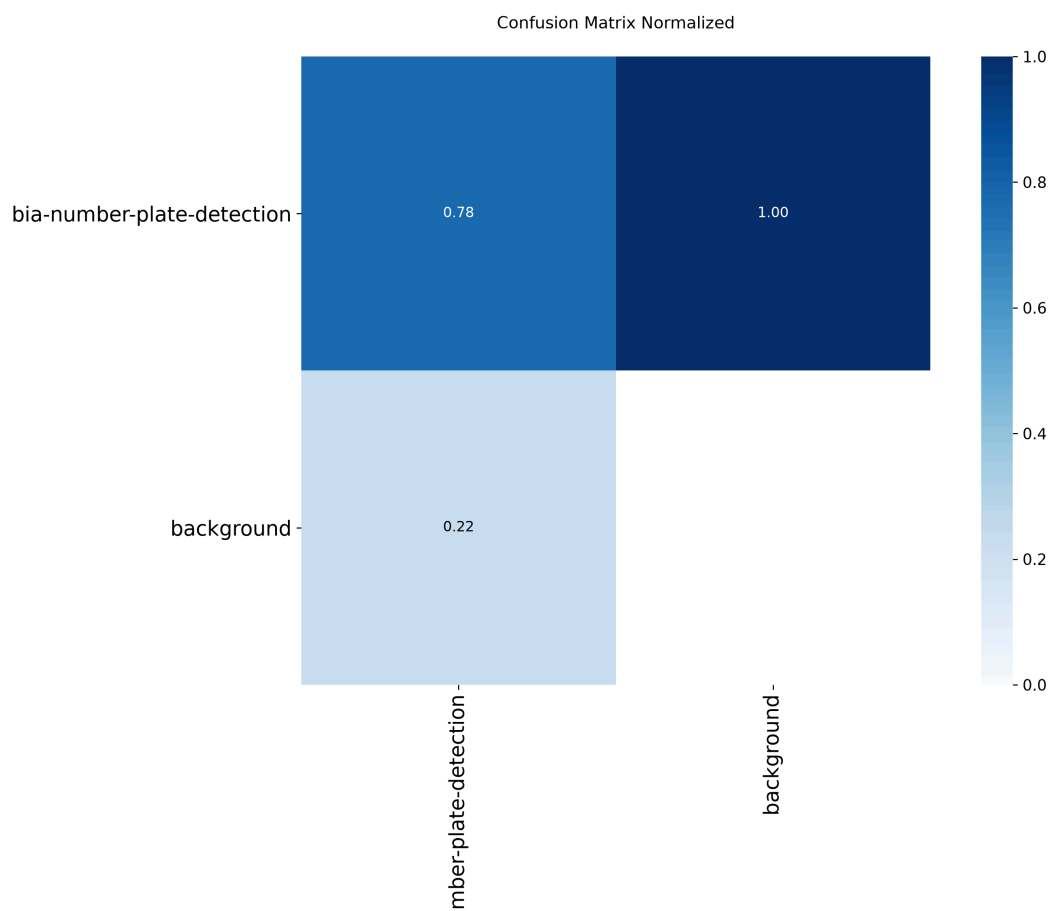
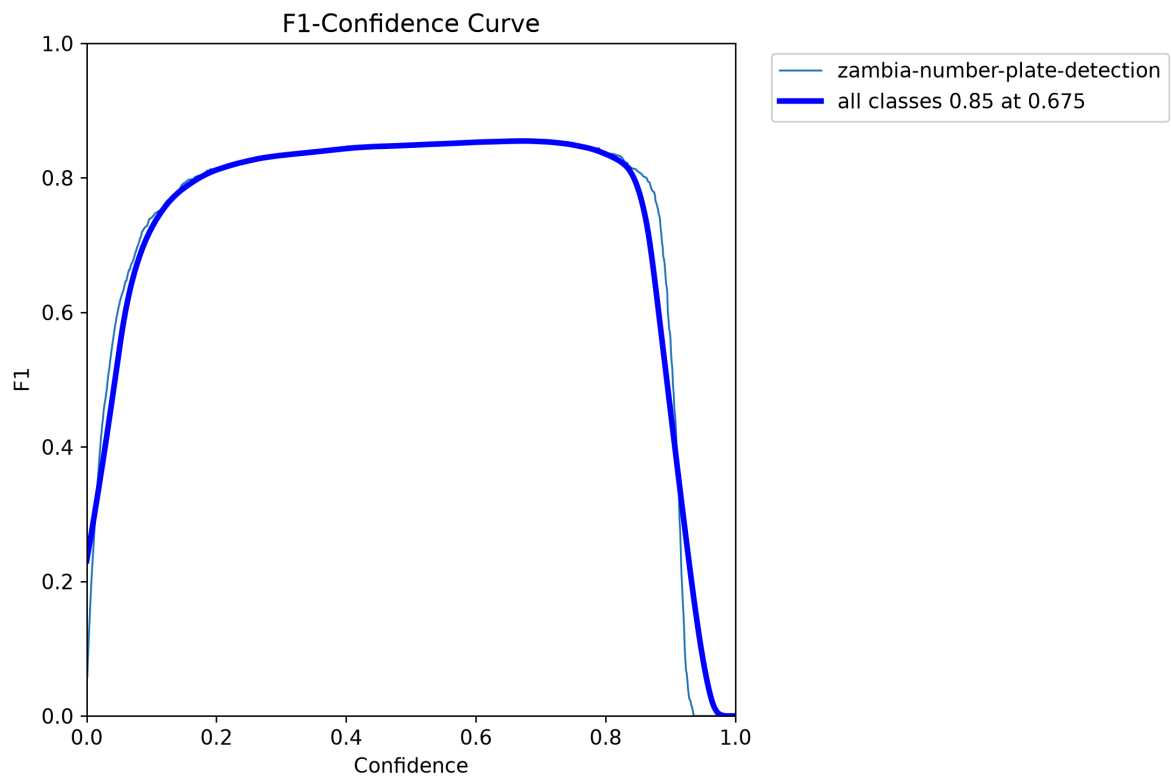
tests/test_plate_utils.py::test_clean_plate_text_strips_punctuation_and_uppercases
PASSED
tests/test_plate_utils.py::test_clean_plate_text_rejects_too_short_or_too_long
PASSED
tests/test_plate_utils.py::test_clean_plate_text_rejects_non_alphanumeric_only_input
PASSED
tests/test_plate_utils.py::test_exact_repeat_within_cooldown_is_a_duplicate PASSED
tests/test_plate_utils.py::test_ocr_jitter_variants_are_treated_as_duplicates
PASSED
tests/test_plate_utils.py::test_genuinely_different_plate_is_not_a_duplicate
PASSED
tests/test_plate_utils.py::test_expired_entries_no_longer_count_as_duplicates
PASSED
tests/test_plate_utils.py::test_expired_entries_are_pruned PASSED

===== 8 passed in 0.06s =====
```

(Run again during the preparation of this report. The original capture is in thesis/assets/test-results/node-pytest-output.txt.)

D.2 Further training evidence

Figures 6.2 to 6.6 in Chapter 6 are the training and evaluation plots. The precision and recall curves as a function of confidence (BoxP_curve.png, BoxR_curve.png) and the F1 curve (BoxF1_curve.png) are in thesis/assets/detection-model/.



Appendix E: Data Collection Instruments

Not applicable. No questionnaires, interviews or surveys were used. The only data collected were plate photographs, described in §3.3.

Appendix F: Additional Code / Configuration

F.1 Configuration template (vrrs-node/.env.example)

```
PHONE_STREAM_URL=http://192.168.1.100:8080/video
BACKEND_URL=http://127.0.0.1:8000
CAMERA_ID=PHONE-NODE-1
LOCATION_SPOTTED=Unknown
MODEL_PATH=C:\Users\user\runs\detect\svds-plate-detector-final-2\weights\best.pt
DETECT_CONF=0.4
PROCESS_EVERY_N_FRAMES=5
RESEND_COOLDOWN_SECONDS=15
```

F.2 Reporting function (vrrs-node/plate_node.py)

```
def report_plate(plate: str, confidence: float) -> None:
    payload = {
        "license_plate": plate,
        "camera_id": CAMERA_ID,
        "confidence_score": confidence,
        "location_spotted": LOCATION_SPOTTED,
    }
    try:
        resp = requests.post(f"{BACKEND_URL}/alerts/check-plate", json=payload,
                             timeout=5)
        resp.raise_for_status()
        print(f"[{plate}] backend status: {resp.json().get('status')}")
    except requests.RequestException as e:
        print(f"[{plate}] failed to reach backend: {e}")
```

F.3 Stream loss recovery (vrrs-node/plate_node.py)

```
ok, frame = cap.read()
if not ok:
    print("Lost connection to stream, retrying...")
    time.sleep(1)
    cap.release()
    cap = cv2.VideoCapture(STREAM_URL)
    continue
```

Appendix G: Other Supporting Material

G.1 Group role split

Role	Work areas	Focus
1. AI/vision	Detection model; camera node	This report
2. Backend	Backend API and database; security and access control	FastAPI, PostgreSQL, /alerts/check-plate, JWT, bcrypt, roles, audit log
3. Real-time and integration	Real-time alerts and monitoring; phone-to-server wiring	WebSocket, camera status, live-feed proxy, system health
4. Frontend	Frontend portals; analytics, testing and documentation	React pages, analytics, run scripts

G.2 Source locations

Item	Path
Camera node	vrrs-node/plate_node.py, vrrs-node/plate_utils.py, vrrs-node/tests/
Trained weights	runs/detect/svds-plate-detector-final-2/weights/best.pt
Training record	thesis/assets/detection-model/ (args.yaml, results.csv, plots)
Dataset metadata	thesis/assets/dataset/ (data.yaml, Roboflow README)
Evaluation script and outputs	thesis/report-2026/assets/